[BURAYA BAŞLIK YAZ: Örn. Schema-Aware Instruction Tuning for Text-to-SQL Semantic Parsing]

[Birinci Yazar Adı Soyadı][1] and [İkinci Yazar Adı Soyadı][1]

Department of Computer Engineering, Izmir Institute of Technology (IYTE), Izmir, Turkey
{[ogrenci1_eposta], [ogrenci2_eposta]}@iyte.edu.tr

Abstract. Text-to-SQL semantic parsing is the task of mapping natural language questions to executable SQL queries, enabling non-expert users to query relational databases. In this work, we fine-tune [MODEL ADI, örn. Flan-T5-base] with instruction tuning on the Spider benchmark dataset and evaluate three schema serialization strategies for representing database structure. We compare our approach against two baselines: (1) zero-shot prompting of a pre-trained language model and (2) [İKİNCİ BASELINE ADI]. Experimental results show that our proposed method achieves [X]% execution accuracy on the Spider development set, outperforming the zero-shot baseline by [Y] percentage points. Ablation studies reveal that the choice of schema encoding format significantly affects model performance, with [FORMAT ADI] yielding the best results. We also provide a qualitative error analysis categorizing failure cases into schema linking errors, incorrect JOIN conditions, and aggregation mistakes.

Keywords: Text-to-SQL · Semantic Parsing · Instruction Tuning · Schema Encoding · Spider Benchmark · Natural Language Interface to Databases

1 Introduction

Natural language interfaces to databases (NLIDB) allow users to query structured data using plain English, eliminating the need for SQL expertise [?]. Text-to-SQL parsing, which maps natural language questions to executable SQL queries, is a central challenge in this domain and has seen significant progress with the advent of large pre-trained language models [?].

Despite recent advances, Text-to-SQL remains challenging due to (1) the compositional nature of SQL, (2) the need to correctly link question entities to database schema elements (schema linking), and (3) the difficulty of generalizing across unseen database schemas [?]. Cross-domain benchmarks such as Spider [?] have highlighted these challenges by evaluating models on databases not seen during training.

In this paper, we make the following contributions:

- We fine-tune [MODEL ADI] with instruction tuning on the Spider benchmark, incorporating explicit schema encoding to help the model understand database structure.
- We systematically compare three schema serialization formats and analyze their impact on execution accuracy via ablation study.
- We evaluate our approach against two baselines — zero-shot prompting and [BASELINE 2 ADI] — using execution accuracy and exact match metrics.
- We provide a detailed error analysis of failure cases, categorized by error type, along with a discussion of ethical considerations.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes our methodology. Section 4 presents the experimental setup. Section 5 reports results. Section 6 provides error analysis. Section 7 discusses limitations, and Section 8 concludes the paper.

2 Related Work

2.1 Sequence-to-Sequence Approaches

Early neural approaches to Text-to-SQL adopted sequence-to-sequence architectures trained to map question–schema pairs to SQL queries [?]. [BU PARAGRAFI KENDI ARAŞTIRMAN İLE GENIŞLET — 3-4 cümle, atıflı]

2.2 Pre-trained Language Models for Text-to-SQL

The introduction of large pre-trained language models has substantially advanced Text-to-SQL performance. [BU PARAGRAFI KENDI ARAŞTIRMAN İLE GENIŞLET — 4-5 cümle, atıflı]

2.3 Schema Encoding and Linking

Effective schema encoding is critical for cross-domain generalization. [BU PARAGRAFI KENDI ARAŞTIRMAN İLE GENIŞLET — 3-4 cümle, atıflı]
Positioning of This Work. Unlike prior approaches that rely on complex graph-based schema linking, we adopt a simpler instruction-tuning framework and systematically compare schema serialization strategies, which to our knowledge has not been directly studied in the context of [MODEL ADI].

3 Methodology

3.1 Problem Formulation

Given a natural language question q and a relational database schema $\mathcal{S} = \{T_1, T_2, \ldots, T_n\}$ where each table T_i contains a set of columns $C_i = \{c_{i,1}, \ldots, c_{i,m}\}$, the goal of Text-to-SQL parsing is to generate an executable SQL query y such that executing y on the target database $\mathcal{D}$ returns the correct answer to q:

$$y^* = \arg\max_y P(y \mid q, \mathcal{S}; \theta) \tag{1}$$

where θ denotes the model parameters.

3.2 Schema Serialization Strategies

A key challenge in Text-to-SQL is representing the database schema as text for input to a language model. We investigate three serialization formats:
Format A — Plain text listing:

Listing 1.1. Format A: Plain text schema representation

```
Table: students | Columns: id, name, age, gpa
Table: courses  | Columns: id, title, credits
FK: enrollments.student_id -> students.id
```

Format B — CREATE TABLE syntax:

Listing 1.2. Format B: CREATE TABLE schema representation

```
CREATE TABLE students (
  id INT PRIMARY KEY,
  name TEXT, age INT, gpa FLOAT
);
```

Format C — Compact notation:

Listing 1.3. Format C: Compact schema representation

```
students(id[PK], name, age, gpa) |
courses(id[PK], title, credits)
```

Format B mimics the input seen during code-oriented pre-training and is expected to align best with code-focused models, while Format A prioritizes readability and Format C minimizes input token count.

3.3 Model Architecture and Fine-Tuning

We fine-tune **[MODEL ADI, örn. google/flan-t5-base]** [?] using the instruction tuning paradigm. The model is an encoder-decoder Transformer with [X] parameters, pre-trained on a mixture of tasks including code generation and question answering.

Our input instruction template is formatted as follows:

Listing 1.4. Instruction template used for fine-tuning

```
### Task: Convert the natural language question to SQL.
### Schema: {serialized_schema}
### Question: {question}
### SQL:
```

To reduce memory requirements, we apply Low-Rank Adaptation (LoRA) [?] with rank $r = [DEER]$, $\alpha = [DEER]$, targeting the {`q_proj`, `v_proj`} attention weight matrices. This reduces trainable parameters from [TOPLAM] to [LORA] million.

3.4 Baseline Methods

Baseline 1 — Zero-shot prompting: We evaluate [MODEL, örn. Mistral-7B-Instruct] in a zero-shot setting, providing only the schema and question in the prompt without any fine-tuning. This establishes the ceiling achievable by prompting alone.
Baseline 2 — [BASELINE ADI]: [BU BASELINE'IN NE YAPTIĞINI 2-3 CÜMLE AÇIKLA. Örn. PICARD kullanıyorsan: "PICARD augments a T5 model with a SQL parser that constrains beam search at inference time to only produce syntactically valid SQL..." atıf ver.]

4 Experimental Setup

4.1 Datasets

We conduct experiments on the **Spider** [?] benchmark, a large-scale cross-domain Text-to-SQL dataset containing [8,659] training examples, [1,034] development examples across [200] databases with [138] different domains. [GeoQuery kullanıyorsanız buraya ekle: We additionally evaluate on GeoQuery [?], a standard semantic parsing benchmark with [880] natural language questions over a US geography database.]

4.2 Evaluation Metrics

We evaluate using two standard metrics: (1) **Execution Accuracy (EX)**: the fraction of predicted SQL queries that return the correct answer when executed on the target database; and (2) **Exact Match (EM)**: the fraction of predictions that exactly match the gold SQL query after normalization (lowercasing, keyword ordering).

4.3 Implementation Details

All models are implemented using the HuggingFace Transformers library [?]. Fine-tuning is performed for [EPOCH SAYISI] epochs with a batch size of [DEĞER], using the AdamW optimizer [?] with a learning rate of [ÖRN. 5e-5] and cosine learning rate scheduling. Experiments are run on [DONANIM, örn. a single NVIDIA A100 40GB GPU / Google Colab Pro with T4 GPU]. Maximum input sequence length is set to [512] tokens and output length to [256] tokens.

5 Results

5.1 Main Results

Table 1 reports execution accuracy and exact match on the Spider development set for all compared methods.

Table 1. Comparison of methods on the Spider development set. EX = Execution Accuracy (%), EM = Exact Match (%). Bold indicates best result.

Method	EX (%)	EM (%)
Baseline 1: Zero-shot [MODEL]	[DEĞER]	[DEĞER]
Baseline 2: [BASELINE ADI]	[DEĞER]	[DEĞER]
Ours: [MODEL] + Format A	[DEĞER]	[DEĞER]
Ours: [MODEL] + Format B	[DEĞER]	[DEĞER]
Ours: [MODEL] + Format C	[DEĞER]	[DEĞER]
Ours (Best)	**[DEĞER]**	**[DEĞER]**

Our proposed fine-tuned model with [FORMAT] schema serialization achieves the highest execution accuracy of [X]%, outperforming the zero-shot baseline by [Y] percentage points and [BASELINE 2] by [Z] points. [İKİ VEYA ÜÇ CÜMLE DAHA — neden bu format daha iyi çalıştı?]

5.2 Ablation Study: Schema Serialization

The ablation results in Table 1 demonstrate that schema serialization format has a measurable impact on model performance. Format B (CREATE TABLE syntax) yields [AÇIKLAMA]. Format C (compact notation) reduces input length by [X]% on average but shows [AÇIKLAMA — daha düşük mü, daha yüksek mi performans?]. [Bu gözlemleri 3-4 cümle detaylandır.]

5.3 Effect of Training Data Size

[İSTEĞE BAĞLI: Farklı eğitim veri büyüklüklerinde performans analizi.]

6 Error Analysis

We manually analyzed [N] prediction errors from the Spider development set to identify systematic failure patterns. Table 2 summarizes the distribution of error categories.

Schema linking errors account for [X]% of failures. These occur when the model selects a plausible but incorrect table. For example, given the question

Table 2. Distribution of error categories in [N] failure cases.

Error Category	Count	%
Schema linking (wrong table)	[DEĞER]	[DEĞER]
Incorrect JOIN condition	[DEĞER]	[DEĞER]
Aggregation error (COUNT/SUM)	[DEĞER]	[DEĞER]
Nested query failure	[DEĞER]	[DEĞER]
WHERE clause string mismatch	[DEĞER]	[DEĞER]
Other	[DEĞER]	[DEĞER]
Total	[N]	100.0

"How many students are enrolled in each department?", the model queries the `students` table directly rather than joining through `enrollments`.
Aggregation errors arise primarily with nested aggregations such as `HAVING COUNT(*) > 1` and `AVG` over grouped results. [BU KATEGORI İÇİN BİR ÖRNEK EKLE]

[DİĞER KATEGORİLERİ BENZER ŞEKİLDE AÇIKLA]

7 Discussion

7.1 Limitations

Our study has several limitations. First, due to computational constraints, we fine-tuned only the [MODEL, örn. base (250M parameter)] variant; larger models may yield substantially better results. Second, our evaluation is limited to the Spider benchmark; performance on domain-specific databases (e.g., medical, legal) may differ. [BAŞKA KISITLAMALARI EKLE — 2-3 cümle]

7.2 Ethical Considerations

Deploying Text-to-SQL systems in production introduces several risks.
Hallucination: The model may generate syntactically valid SQL that queries non-existent columns or tables, silently returning incorrect results. Users without SQL expertise cannot detect such errors.
SQL Injection Risk: While the model operates on trusted schema inputs in our setting, integrating user-provided free-text into SQL generation pipelines could introduce injection vulnerabilities if outputs are not sanitized.
Domain Bias: Spider's training data skews toward English and standard business domains (universities, music, sports). Models trained on Spider may perform poorly on multilingual queries or specialized domains such as healthcare or legal databases, raising fairness concerns for non-English-speaking or domain-specific users.

8 Conclusion

In this paper, we presented a Text-to-SQL semantic parsing system based on instruction-tuned [MODEL ADI] with systematic comparison of schema serialization strategies.

Our experiments on the Spider benchmark demonstrate that fine-tuning with structured schema encoding significantly outperforms zero-shot prompting, achieving [X]% execution accuracy on the development set. Among the three serialization formats tested, [FORMAT B] yields the best performance, suggesting that [AÇIKLAMA]. Error analysis reveals that schema linking and nested query generation remain the primary bottlenecks.

Future work could explore (1) graph-based schema linking modules, (2) retrieval-augmented generation to provide few-shot SQL examples at inference, and (3) extending evaluation to low-resource languages to address the bias limitations identified in this study.

References